

UNDERSTAND IT · RUN IT · EXPLAIN IT · 9 SEPTEMBER 2026

mutagent & mirage

Two projects, one idea, explained from zero

Written for someone who has not seen either project before.
Nothing is assumed. Every term is defined the first time it appears,
every screen you will see is shown in full, and every command tells
you what should happen.

Author Naga Leela Krishna Rajamahendrarapu

mutagent Java 21 · 83 tests · github.com/leelakrishna288/mutagent

mirage Python 3.14 · 28 tests · D:\AI\Projects\mirage

Both verified on your machine, 9 September 2026

Before you start

IF YOU READ ONLY ONE PAGE, READ THIS ONE

You built two tools. They do different jobs, but they are the **same idea** applied twice:

You cannot tell whether a machine's answer is any good by looking at it. You have to break something on purpose and see whether the machine notices.

	mutagent	mirage
Language	Java	Python
Job	Are these software tests any good?	Are these AI answers really coming from the documents?
Breaks	the <i>code</i>	the <i>documents</i>
Asks	did any test fail?	did the answer change?
If nothing happens	the tests are decoration	the answer was invented

That is the whole thing. Everything else in this document is detail.

How this document is arranged

- A** Before you start — the one idea, and the words you need
- B** mutagent, from zero — what a test is, why coverage lies, what it does
- C** mirage, from zero — what RAG is, why citations prove nothing, what it does
- D** Running both — every command, and exactly what you should see
- E** Reading the output — every line of both reports, explained
- F** Explaining them — 10 seconds, 30 seconds, two minutes, and the hard questions
- G** Quick reference — one page to keep open

A1 The eight words you need

Everything else is built out of these. Read them once; they are repeated in context later.

Word	What it means
Unit test	A small program that runs your code and checks the answer. If the answer is wrong, the test <i>fails</i> . Failing is the test doing its job.
Assertion	The line inside a test that does the checking, like <code>assertEquals(10, discount)</code> . A test with no assertion can never fail.
Coverage	The percentage of your code that was <i>run</i> while the tests ran. Note the word: <i>run</i> , not <i>checked</i> .
Mutant	A copy of your code with one small thing deliberately broken. Like changing <code>>=</code> to <code>></code> .
Killed / Survived	You run the tests against the mutant. If a test fails, the mutant is killed — good, your tests noticed. If every test still passes, the mutant survived — bad, a real bug would have slipped through.
LLM	Large Language Model. The thing behind ChatGPT and Claude. It writes text. It is often right and sometimes confidently wrong.
RAG	Retrieval-Augmented Generation. Search your documents first, then give the results to an LLM and ask it to answer using them. It is how almost every company "AI assistant" works.
Chunk	Documents are too big to hand to an LLM whole, so they are cut into small pieces. Each piece is a chunk. Search finds chunks, not whole documents.

THE ONE THING PEOPLE GET WRONG ABOUT COVERAGE

A test that runs every line of your code and checks **nothing** scores 100% coverage. Coverage measures which lines *executed*. It cannot tell a real test from a test that just calls the method and throws the answer away. That gap is why mutagent exists.

B mutagent, from zero

B1 Start with a real piece of code

This is a real file from your project: `fixtures/calc/src/calc/Pricing.java`. It works out a discount percentage for an order. Read it slowly — the whole of mutagent hangs off one line of it.

```
public int discountPercent(int quantity, boolean member) {
    if (quantity < 0) {
        throw new IllegalArgumentException("quantity must not be negative");
    }
    int discount = 0;
    if (quantity >= BULK_THRESHOLD) {           // BULK_THRESHOLD = 10
        discount = 10;
    }
    if (quantity >= WHOLESALE_THRESHOLD) {     // WHOLESALE_THRESHOLD = 50  <-- line 19
        discount = 20;
    }
    if (member) {
        discount = discount + 5;
    }
    if (discount > MAX_DISCOUNT) {           // MAX_DISCOUNT = 25
        discount = MAX_DISCOUNT;
    }
    return discount;
}
```

In plain English: order 10 or more, get 10% off. Order 50 or more, get 20% off. Members get another 5%. Never more than 25% total.

B2 Now the tests — and why they are worthless

This is the real test file, `fixtures/calc/test/calc/PricingTest.java`:

```
@MiniTest
public void smallOrderHasNoDiscount() {
    Assert.assertEquals(0, pricing.discountPercent(5, false), "small order discount");
}

@MiniTest
public void totalIsQuantityTimesUnitPrice() {
    Assert.assertEquals(500L, pricing.total(5, 100L, false), "undiscounted total");
}

@MiniTest
public void largeOrderGetsSomeDiscount() {
    Assert.assertTrue(pricing.discountPercent(60, false) > 0, "large order discount");
}
```

Three tests. They all pass. They run every line of the method. **Coverage looks fine.**

Now look at what quantities they actually use: **5** and **60**. The thresholds in the code are **10** and **50**. The tests never once try a quantity *at* a threshold.

HERE IS THE BUG THESE TESTS CANNOT CATCH

Change line 19 from `quantity >= 50` to `quantity > 50` . One character.

Now a customer ordering **exactly 50** units gets 10% instead of 20%. That is a real money bug.

Run the tests. **All three still pass.** Test one uses 5 — unaffected. Test three uses 60 — still above 50, still gets a discount, still passes. Nobody tested 50.

This is the entire problem. The tests look fine, the coverage looks fine, and the suite would let that bug reach production. **Nothing existing tells you this.**

B3 So: break the code on purpose

That one-character change — `>=` to `>` — is a **mutant**. mutagent makes hundreds of them automatically, in six families:

Family	Example change
Conditional boundary	<code>>=</code> → <code>></code> , <code><</code> → <code><=</code>
Negate conditionals	<code>==</code> → <code>!=</code>
Boolean connector	<code>&&</code> → <code> </code>
Boolean return value	<code>return true</code> → <code>return false</code>
Math operator	<code>+</code> → <code>-</code> , <code>*</code> → <code>/</code>
Increment / decrement	<code>i++</code> → <code>i--</code>

For each mutant it runs your tests and asks one question:

```
Did any test fail?
YES  -> KILLED    good. your tests would have caught this bug.
NO   -> SURVIVED  bad.  this bug would reach production.
```

Mutation score = killed ÷ total. On this fixture: **20%**. The suite catches one defect in five. Coverage said everything was fine.

B4 Where the AI comes in

Finding the holes is useful. Someone still has to write the missing tests, and that is where the work dies in most companies. So mutagent asks an LLM to write them — but it does **not** trust what comes back.

Each proposed test must pass four machine checks, in order:

#	Check	Why
1	Policy check (before anything runs)	Is it trying to overwrite production code? Touch the filesystem, network, or reflection? Rejected without ever being compiled.
2	Does it compile?	An LLM produces code that does not compile more often than people admit.
3	Does it pass on the unmutated code?	If a test fails against correct code, the test is wrong, not the code.
4	Does it kill a mutant that survived before?	The one that matters. A test that compiles and passes but catches nothing is worthless.

THE RESULT YOU CAN SHOW ANYONE

mutagent asks for **three** candidate tests per round and scores them. On this fixture: one killed 7 mutants (score **73.32**), one compiled and passed and killed nothing (**3.63** — worthless), one asserted something untrue (**-1.29**). Only the first was kept. Mutation score went **20% → 90%**.

An unreliable writer became useful *without becoming reliable*, because a compiler and a test run decide, not an opinion.

B5 What is actually inside it

Six layers, each only allowed to know about the ones below it:

ENTRY POINTS	the command line; an MCP server so an AI client can call it as a tool
THE AGENT	the loop, the safety checks, the scoring, the budgets
MODEL ACCESS	one interface, four implementations (incl. an offline "replay" model)
MUTATION	plan the mutants, run them, record what happened
COMPILE + RUN	compiles Java in memory using the JDK's own compiler; runs the tests
VALUE TYPES	the plain data records everything else passes around

Three things worth knowing about that stack:

- **Nothing is written to your disk during a run.** Java is compiled in memory using the JDK's own compiler, called from inside the running program.
- **Zero third-party libraries.** Maven Central was blocked in the build environment, so the mutation engine, the compile harness, the JSON codec, the test runner and the MCP server were all written from scratch. It builds with nothing but a JDK.
- **The agent layer is the only part that knows an LLM exists.** That is why the mutation engine can be tested completely without one — and is.

C mirage, from zero

C1 What RAG is, in one example

Your company has 10,000 internal documents. You want staff to ask questions in plain English. So you build this:

```
Someone asks: "What was our revenue last year?"
|
v
1. SEARCH the documents -> finds the 4 most relevant chunks
|
v
2. PASTE those chunks into a message to an LLM, along with the question,
   saying "answer using only this"
|
v
3. The LLM writes: "Revenue was 412 million dollars."
|
v
4. Show the answer, plus the document it came from
```

That is **RAG**. Retrieve, then generate. Almost every corporate "AI assistant" is this.

C2 The problem nobody can currently answer

Everyone is happy, because the answer is right and it shows its source. But:

DID IT ACTUALLY READ THAT DOCUMENT?

LLMs have memorised colossal amounts of text during training. A model can produce a confident, correct-looking answer **from memory** while the document sits in front of it, unread — and then cite the document because it looked relevant.

And here is the sharp bit: **the citation is also generated text**. The model writes the citation the same way it writes the answer. It is not evidence of anything.

So a *wrong* answer with a real citation looks exactly like a *right* answer with a real citation. Everything teams do today is an *observation*, and none of it is a *test*:

What teams do	Why it doesn't answer the question
"It shows a citation"	Generated text. Proves nothing.
Ask a second LLM "is this grounded?"	A model's opinion about a model's output. Fails in the same direction as the first one.
Measure search accuracy (recall@k)	Tells you the right chunk was fetched . Says nothing about whether the answer used it.
Humans spot-check	Doesn't scale, doesn't repeat, drifts as the system changes.

C3 The idea — the same trick as mutagent

In mutagent you break the **code** and see if the tests notice. Here you break the **documents** and see if the answers notice.

Worked example, using the real fixture in your repo. The document says:

```
"Northwind Logistics reported total revenue of 412 million dollars  
for the fiscal year 2025."
```

You ask: "What was Northwind Logistics total revenue in fiscal year 2025?"

It answers: **412 million dollars**. Correct, and it cites that document.

Now mirage intervenes:

What it does	Answer changes?	What that means
Changes 412 to 831 in the document	says 831 → KILLED	good — it really is reading it
Changes 412 to 831 in the document	still says 412 → SURVIVED	the number did not come from your document
Deletes the document entirely	says "I don't know" → KILLED	correct behaviour
Deletes the document entirely	still answers 412 → SURVIVED	it was never reading it at all

That last row is the strongest finding the tool can produce. With the document gone there is no other place the answer could have come from.

C4 The seven mutations

#	Name	What it does	If the answer doesn't change
1	Evidence deletion	deletes the supporting chunks	strongest finding
2	Fact corruption	changes every number	figure didn't come from the document
3	Entity swap	replaces names	subject came from memory
4	Contradiction	adds a chunk saying the opposite	conflicting evidence ignored
5	Distractor	adds a similar-but-wrong chunk	search isn't discriminating
6	Paraphrase (control)	says the same thing in different words	correct — it <i>should</i> hold
7	Irrelevant corruption (control)	corrupts an unrelated chunk	correct — it <i>should</i> hold

Why 6 and 7 are the clever part

Without them the score can be gamed. A system that changes its answer to **everything** would score 100% and be useless — it is not grounded, it is unstable.

- **Paraphrase (control 6):** if rewording the source changes the answer, the system is **unstable**, not grounded.
- **Irrelevant corruption (control 7):** if corrupting a document with nothing to do with the question changes the answer, the system is **noisy**, and the whole score is untrustworthy.

If either control fails, mirage refuses to print a score at all and says INCONCLUSIVE. A number you cannot trust is worse than no number.

C5 The hardest problem, and how it is solved

LLMs are random. Ask the same question twice and the wording differs slightly. So when a mutated answer differs from the original — **is that the mutation working, or the model being random?**

You cannot know unless you measure the randomness first. So before any mutation, mirage asks each question **five times with nothing changed** and measures how much the answer moves on its own. Then the paraphrase control refines it further. The rule becomes:

THE RULE

An answer only counts as "changed" if it moves **further than a paraphrase of its own evidence moves it.**

This was not designed on paper — the first version of the tool failed its own paraphrase control at 50% and reported INCONCLUSIVE. The reference reader echoes the source's wording, so rewording the source reworded the answer, and the tool called that "a real change". The fix is in `calibrate.py` and is pinned by a test. **It is the best thing you can talk about on this project.**

C6 What is inside it

<code>corpus.py</code>	documents -> chunks; snapshot/restore so mutations always undo
<code>embed.py</code>	turns text into numbers, with a cache so one mutated chunk does not force the whole corpus to be redone
<code>retrieve.py</code>	keyword search (BM25) + meaning search (dense), combined
<code>llm.py</code>	the model, behind an interface: offline replay, or Anthropic
<code>pipeline.py</code>	the RAG pipeline itself, behind an interface
<code>evidence.py</code>	works out which chunks an answer actually rests on
<code>operators.py</code>	the seven mutations
<code>calibrate.py</code>	the two-stage threshold described above
<code>compare.py</code>	did the answer change? a ladder, not string equality
<code>engine.py</code>	the run
<code>report.py</code>	console + HTML output
<code>cli.py</code>	mirage scan mirage test

Two details worth knowing:

- **It is not tied to any framework.** Anything that can answer a question and say which chunks it retrieved can be measured — a LangChain app, your own Python, or a Java service over HTTP.
- **Evidence *sets*, not single chunks.** If a second document also states the fact, corrupting only the first would produce a false alarm. mirage finds every chunk carrying the claim and mutates them together.

D Running both

Everything below has been run on your machine and produces exactly what is shown. Neither project needs an API key, an internet connection, or any money.

D1 What is already set up

Thing	Where / status
Java (for mutagent)	JDK 21.0.12.1 — already installed and on PATH
Python (for mirage)	3.14.6 at C:\Users\leela\AppData\Local\Programs\Python\Python314
mutagent	D:\AI\Projects\mutagent — built, 83 tests passing
mirage	D:\AI\Projects\mirage — installed in its own venv, 28 tests passing

ONE THING THAT WILL CONFUSE YOU

Typing `python` in PowerShell gives an error about the Microsoft Store. That is a Windows placeholder shadowing your real Python. It does **not** mean Python is missing. For mirage, always use the copy inside the project: `.\venv\Scripts\python.exe`.

D2 mutagent — the five commands

1. Build it

```
cd D:\AI\Projects\mutagent
.\build.ps1
```

You should see: `built -> out\ (59 source files)` . Takes about 15 seconds. This compiles the Java. Re-run it after any code change.

2. Run its own tests

```
.\test.ps1
```

You should see: `tests run: 83 then all passing` . This is the "is the project healthy" command. Run it before and after any change.

3. Find the holes — the important one

```
java -cp out dev.mutagent.cli.Main report fixtures\calc
```

You should see a list of surviving mutants and `MUTATION SCORE 20.0%` . This is the `Pricing.java` example from Part B. Every line in that list is a change to the code that no test noticed.

4. Explain one hole in English

```
java -cp out dev.mutagent.cli.Main explain fixtures\calc --id M003
```

You should see a plain-English paragraph about line 19, showing the original and mutated line side by side. **No AI is involved** — this is worked out from the operator and the position, so it says the same thing every time.

5. Close the holes — the demo

```
java -cp out dev.mutagent.cli.Main harden fixtures\calc
```

You should see: three candidate tests scored, the best one accepted, and mutation score 20.0% -> 90.0% .

WANT THE WHOLE TOUR IN ONE COMMAND?

`.demo.ps1` runs all of the above in sequence with headings, plus the benchmark. About four minutes. This is what to run when showing someone.

D3 mirage — the four commands

Every mirage command starts by moving into the project:

```
cd D:\AI\Projects\mirage
```

1. Run its own tests

```
.\.venv\Scripts\python.exe -m pytest -q
```

You should see: 28 passed .

2. The grounded run — a system that IS reading its documents

```
.\.venv\Scripts\mirage.exe scan fixtures\false_confidence
```

You should see: controls PASS, and EVIDENCE SENSITIVITY 60.0% .

3. The ungrounded run — THE DEMO

```
.\.venv\Scripts\mirage.exe scan fixtures\false_confidence --ungrounded
```

You should see: EVIDENCE SENSITIVITY 30.0% , and near the top a survivor labelled `evidence_deletion` .

RUN 2 AND 3 BACK TO BACK. THAT IS THE DEMO.

Same documents. Same questions. Same citations. In the second one the model answers partly from memory — and mirage catches it by **deleting the source document and seeing the answer stay the same**. Score halves.

4. The build gate

```
.\.venv\Scripts\mirage.exe test fixtures\false_confidence --threshold 0.60
```

You should see: GATE: PASS 60.0% vs threshold 60% . Add `--ungrounded` and it says GATE: FAIL and stops the build. That is the difference between a research script and a tool a team would install.

D4 If something goes wrong

What you see	What to do
python : The term 'python' is not recognized or a Microsoft Store popup	Expected. Use <code>.\.venv\Scripts\python.exe</code> instead.
running scripts is disabled on this system	<code>Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass</code> — applies to that window only.
javac is not recognized	Open a new PowerShell window. The old one has a stale PATH.
BASELINE IS NOT GREEN - stopping	Working as designed. The project's own tests fail, so mutation results would be meaningless. Fix those first.
mirage says INCONCLUSIVE	Also working as designed — a control failed, so no score is trustworthy. See §C4.
OutOfMemoryError on a big project	Give it more memory: <code>java -Xmx4g -cp out ...</code>

E Reading the output

The reports look intimidating. They are not. Here is every line of both, explained.

E1 mutagent's report

```
calc.Pricing
M001 risk 100 (HIGH) line 12 discountPercent() Conditional boundary: '<' -> '<='
M004 risk 57 (MEDIUM) line 23 discountPercent() Math operator: '+' -> '-'
```

Read each line left to right:

M001	the mutant's id — use it with the <code>explain</code> command
risk 100 (HIGH)	how much this one should worry you, 0–100. Worked out from how severe the change is, whether it sits in an <code>if</code> , whether the method is public, and how long the method is. It is a judgement, not a measurement — and the README says so.
line 12	where in the file
discountPercent()	which method
the rest	exactly what was changed

Then the metrics block:

```
METRICS
planned          10
not viable       0  (mutated source did not compile; excluded)
infrastructure    0  (engine failures; excluded)
viable           10
killed            2
survived          8
MUTATION SCORE   20.0%
compilation       0 full, 10 single-class, 2 cache reuses
coverage selection 11 test executions skipped (52%)
```

planned	how many mutants it made
not viable	the broken code didn't compile. Excluded — a change that can't compile isn't a bug a test could have caught.
infrastructure	the tool itself failed. Also excluded — but reported, not hidden , so a broken run can't masquerade as a good score. If this is not 0, something went wrong.
viable	the honest denominator
killed	a test failed — the bug was caught
survived	every test still passed — each of these is a proven hole
MUTATION SCORE	killed ÷ viable. 20% on a suite with full coverage. That gap is the whole point.
compilation	proof the cache works: ten mutants, zero full rebuilds
coverage selection	proof it only runs tests that can possibly matter

E2 mirage's report, top to bottom

```
baseline answer variance    mean self-similarity 1.000 (k=5, sd=0.000)
difference threshold        0.893 [self-consistency + paraphrase control]
paraphrase floor           0.913 (an answer must move further than a paraphrase does)
```

The system was asked each question 5 times with nothing changed, and gave identical answers every time (1.000). Then the paraphrase control set the bar at 0.893. **An answer only counts as changed if it scores below 0.893.** This block is what makes every number underneath it mean something.

```
CONTROLS - the answer SHOULD hold for these
irrelevant_corruption    4/4 held (100%)
paraphrase               4/4 held (100%)
verdict                  PASS
```

Both sanity checks passed, so the score can be trusted. **If this said FAIL, everything below would be replaced by INCONCLUSIVE.**

```
SURVIVORS - evidence corrupted, answer unchanged. Highest weight first.
M004 w0.60 contradiction      sim 1.000
      Q: What was Northwind Logistics total revenue in fiscal year 2025?
      A: Northwind Logistics reported total revenue of 412 million dollars...
      inject a chunk contradicting the evidence
```

w0.60 is how seriously to take it (deletion is 1.00, distractor 0.40). sim 1.000 means the answer was *identical* after the corruption. Then the question, the answer, and what was done to the documents.

BY OPERATOR			
evidence_deletion	4/4	100.0%	weight 1.00
fact_corruption	4/4	100.0%	weight 0.90
entity_swap	4/4	100.0%	weight 0.85
contradiction	0/4	0.0%	weight 0.60
distractor	0/4	0.0%	weight 0.40

THIS BLOCK IS THE USEFUL PART — IT IS A DIAGNOSIS

The pipeline is **perfect** at the three that matter most: delete the evidence, corrupt the numbers, swap the names — it notices every time. It is **useless** at the last two. Those are different problems with different fixes: contradiction is a **prompt** problem (it isn't told what to do with conflicting sources); distractor is a **search** problem (it can't tell a near-miss from the real thing). A single percentage would have hidden both.

EVIDENCE SENSITIVITY	60.0%	(killed / scored)
weighted sensitivity	73.3%	(deletion counts most)

60% is the plain count. 73.3% weights each mutation by how much it proves — higher here because the pipeline aces the heavy operators and only fails the light ones.

E3 The two numbers to remember

Project	Number	What it means
mutagent	20% → 90%	a suite with full coverage caught 1 defect in 5; after hardening, 9 in 10
mirage	60% vs 30%	same documents, same citations — one system reads them, one doesn't

F Explaining them

F1 Ten seconds

mutagent: "AI writes tests. I break the code on purpose to prove the tests actually work."

mirage: "AI answers from documents. I corrupt the documents to prove it's really reading them."

F2 Thirty seconds

"Code coverage lies — a test with no assertions gets 100% coverage. So I built a tool that changes your code on purpose and checks whether any test notices. Then it asks an LLM to write the missing tests, and keeps one only if re-running the mutations proves it catches something the old suite missed. On the demo, a suite with full coverage scored 20%. After hardening, 90%.

Then I did the same thing for RAG. A RAG system cites its sources, but the citation is generated text — it proves nothing. So mirage deletes the document behind an answer and re-asks. If the answer doesn't change, it was never reading it."

F3 Two minutes, for an engineer

"The thing I care about is the evaluation problem. Everyone can call an LLM; almost nobody can tell you whether what came back was any good. The usual answer is LLM-as-judge, which just asks a second model for an opinion and fails in the same direction as the first.

*So I picked problems where correctness is **mechanically checkable**. For a test, 'is this good' has an objective answer: does it catch a defect the suite missed? That's a mutation re-run — a compiler and a test runner, no model involved. mutagent asks for three candidates, checks them in code before compiling, and keeps the best only if it killed something.*

Then I asked the same question about retrieval, and the answer is the same shape: corrupt the evidence and see if the answer moves. The hard part there isn't the mutation, it's knowing what counts as a change — LLMs are random. So mirage measures the pipeline's own variance first, then uses a paraphrase control to set the threshold. An answer counts as changed only if it moves further than a paraphrase of its own evidence moves it. My first version skipped that and failed its own control at 50%."

F4 The questions you will be asked

Question	Answer
"Isn't mutation testing just PIT?"	"PIT is the engine — and there's an adapter for it behind the same interface. The new part is the closed loop: generate, verify, accept or reject. PIT tells you what's wrong; this fixes it and proves the fix."
"How do you stop the model writing bad tests?"	"I don't stop it, I measure it. Bad ones score low and are discarded. And the guardrails are code, not prompt text — it structurally cannot overwrite production code or touch the filesystem."
"How do you know a mirage difference isn't just randomness?"	The best question, and you have the answer. "I measure the pipeline's own variance first, then let the paraphrase control set the threshold. An answer counts as changed only if it moves further than a paraphrase does."
"Couldn't the answer have come from another document?"	"Yes — that's why I resolve an evidence set and mutate every chunk carrying the claim, not just the top hit. And deletion survivors are weighted highest because with the set gone there's nowhere else it could come from."
"Is this novel?"	"No, and I wouldn't claim that. Perturbation-based grounding evaluation exists in research; RAGAS measures adjacent things. What I built is an automated mutation-testing framework around it — operator taxonomy, two control groups, noise calibration, CI gate."
"Is it reinforcement learning?"	"No. Nothing updates any model weights. It's an evaluation signal used to select among candidates within a run."
"Why no Spring, no libraries in mutagent?"	"Maven Central was unreachable in the build environment. So I wrote the mutation engine, the compile harness, the JSON codec, the test runner and the MCP server. It builds with nothing but a JDK."
"What doesn't work?"	Answer with \$F5, directly. This question is a test of whether you know your own system.

F5 What does not work — know these cold

Project	Limitation
mutagent	Cannot score a real JUnit project end to end — the built-in runner doesn't understand JUnit annotations. Biggest gap; the PIT adapter is the fix.
mutagent	No successful live model run has been recorded. Every demo uses the replay model and says so. Do not describe a live run as having succeeded.
mutagent	The PIT adapter has never been executed — only its report parser is tested.
mutagent	A mutant can leave a thread running that nothing can stop. Java has no safe way to kill a thread; they're counted and the run stops cleanly instead of dying.
mirage	Not novel — see F4.
mirage	A survivor is strong evidence, not a verdict . The answer might have come from a chunk outside the resolved evidence set.
mirage	The embedder is a simple hashing vectoriser, not a trained encoder. It keeps the demo offline; swap it for production.
mirage	The Anthropic adapter has never made a live call.
both	Cost: every mutant is a model call on the live path. The offline replay models exist for exactly this reason.

WHY VOLUNTEERING THESE MAKES YOU STRONGER, NOT WEAKER

A candidate who says "it's perfect" gets probed until it isn't. A candidate who names four limitations before being asked gets believed about everything else. Both READMEs already list these — you are not admitting anything, you are quoting your own documentation.

G Quick reference

Commands

Command	What you get
mutagent — first: <code>cd D:\AI\Projects\mutagent</code>	
<code>.\build.ps1</code>	compiles → <code>built -> out\</code> (59 source files)
<code>.\test.ps1</code>	tests run: 83 / all passing
<code>.\demo.ps1</code>	the full guided tour, ~4 min
<code>java -cp out dev.mutagent.cli.Main report fixtures\calc</code>	the holes + MUTATION SCORE 20.0%
<code>... explain fixtures\calc --id M003</code>	why one hole exists, in English
<code>... harden fixtures\calc</code>	20.0% → 90.0%
mirage — first: <code>cd D:\AI\Projects\mirage</code>	
<code>.\venv\Scripts\python.exe -m pytest -q</code>	28 passed
<code>.\venv\Scripts\mirage.exe scan fixtures\false_confidence</code>	60.0% , controls PASS
<code>... scan fixtures\false_confidence --ungrounded</code>	30.0% — the demo
<code>... test fixtures\false_confidence --threshold 0.60</code>	GATE PASS / FAIL

Numbers

mutagent	mirage
Java 21 · ~5,000 lines · 47 classes	Python 3.14 · ~1,360 lines
83 tests , 10 suites, CI green	28 tests , CI on 3 Python versions
0 third-party dependencies	2 dependencies (numpy, scikit-learn)
calc: 20% → 90%	grounded 60.0% (73.3% weighted)
dogfood: 446 mutants, 25.3%	ungrounded 30.0% (36.7% weighted)
benchmark: 1,138 mutants over 7,379 lines of Apache Commons	7 operators, 2 of them controls
6 mutation operators	gate exits 0 (pass) / 1 (fail) / 2 (inconclusive)

The five sentences

1. Coverage measures which lines *ran*. Mutation testing measures which defects would be *caught*. A suite can have 100% of the first and 20% of the second.
2. A generated test is kept only if it kills a mutant that previously survived. The judge is a compiler and a test run, not another model.
3. A RAG citation is generated text. It proves nothing. Deleting the document and re-asking proves something.
4. An answer counts as changed only if it moves further than a paraphrase of its own evidence moves it — otherwise you are measuring randomness.
5. Guardrails are code, not prompt text. A prompt is a request; a check is a guarantee.

What is still open

Item	Status
mutagent live model run	Needs API credits. Everything else is done.
mirage on GitHub	Not yet pushed. Repo not created.
Resume with both projects	Currently one page, mutagent only. Two pages with both takes ATS keyword coverage from 74% to ~93%.

mutagent & mirage — understand, run, explain. 9 September 2026. Every command in this document was executed on your machine and produces the output shown. Every number was read from the tools' own output, not from memory. Where something has not been verified, \$F5 says so.

D:\AI\Projects\mutagent · D:\AI\Projects\mirage